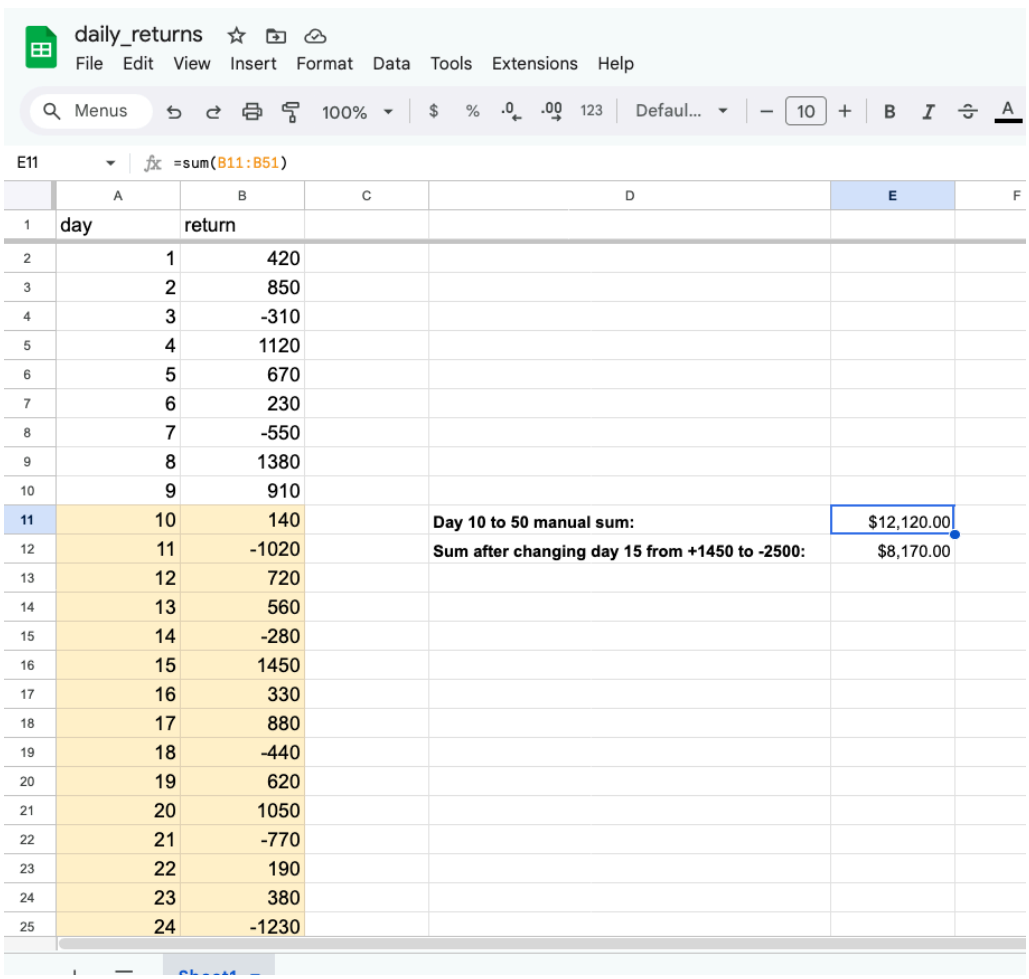
Daniel O'Connell
04/20/2026
CSPB-2270

Code Walkthrough: Fenwick Trees

My code begins with a .csv file that I created in Google Sheets to store simulated investment performance. This csv contains 2 columns: one for the date (stored as a number from 0-250 to represent 250 days of investment data), and the other as a daily investment return as a dollar amount. (Note: I originally built this with the returns as percentages, but due to the formula for compounding returns when considering daily percentage changes, I decided to change to dollars to be more correct and not have to change my implementation to handle multiplication down the tree. This is still a valid way to look at the numbers from an investment management perspective).

To show that this program is performing as expected, I will demonstrate that the program correctly sums from day 10 to day 50, that you can manually update a day's return in the program, and then that the summation with the updated return also correctly reflects the update.

Screenshot from google sheets showing the manual summation before and after update:



	A	B	C	D	E	F
1	day	return				
2	1	420				
3	2	850				
4	3	-310				
5	4	1120				
6	5	670				
7	6	230				
8	7	-550				
9	8	1380				
10	9	910				
11	10	140		**Day 10 to 50 manual sum:**	$12,120.00	
12	11	-1020		**Sum after changing day 15 from +1450 to -2500:**	$8,170.00	
13	12	720				
14	13	560				
15	14	-280				
16	15	1450				
17	16	330				
18	17	880				
19	18	-440				
20	19	620				
21	20	1050				
22	21	-770				
23	22	190				
24	23	380				
25	24	-1230				

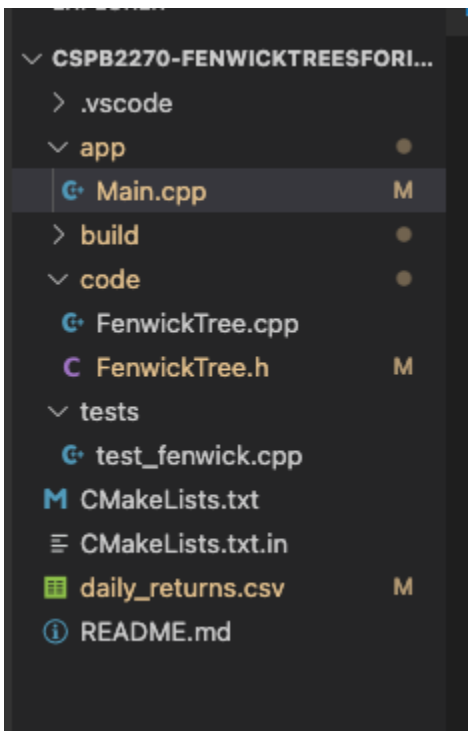
Output from the terminal after running the application:

```
jovyan@jupyter-daoc2270:~/CSPB2270-FenwickTreesForInvestmentPerformance/build$ make && ./run_app
[ 50%] Built target run_tests
Consolidate compiler generated dependencies of target run_app
[ 66%] Building CXX object CMakeFiles/run_app.dir/app/Main.cpp.o
[ 83%] Linking CXX executable run_app
[100%] Built target run_app
Loaded 252 days of returns.
Cumulative return from day 10 to day 50: $12120
Updated day 15 from $1450 to $-2500
Cumulative return from day 10 to day 50 after update: $8170
```

As you can see, the changes are the same in each. Each summed to $12,120 before day 15 was changed, and then summed to $8170 after day 15 was changed. Logically, 12120 - 1450 - 2500 = 8170.

I tried to keep the overall implementation as close in structure to the homeworks as possible.

The csv file is statically uploaded into the repository and is referenced from the Main.cpp file



Main.cpp has a function that loads the csv and stores it in a vector that is then used by the rest of the program.

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include "../code/FenwickTree.h"

using namespace std;

//Read daily returns from a CSV file with "day,return" headers.
//Returns a 0-indexed vector where result[0] = day 1 return.
vector<double> loadCSV(const string& filename) {
    vector<double> returns;

    ifstream file(filename);
    if (!file.is_open()) {
        cout << "Error: could not open file \"" << filename << "\"." << en
        return returns;
    }

    string line;
    getline(file, line); //skip header row

    while (getline(file, line)) {
        stringstream ss(line);
        string dayStr, returnStr;
        getline(ss, dayStr, ','); //parse
        getline(ss, returnStr, ',');
        returns.push_back(stod(returnStr));
    }

    //Output ingested returns
    return returns;
}

int main() {
```

The main() function then takes the filepath to the csv as an argument and creates a vector. If the file cannot be found, an error message is output to the console. The logic for summing from day 10 to 50 is configured in the main() function, utilizing the functions in FenwickTree.cpp

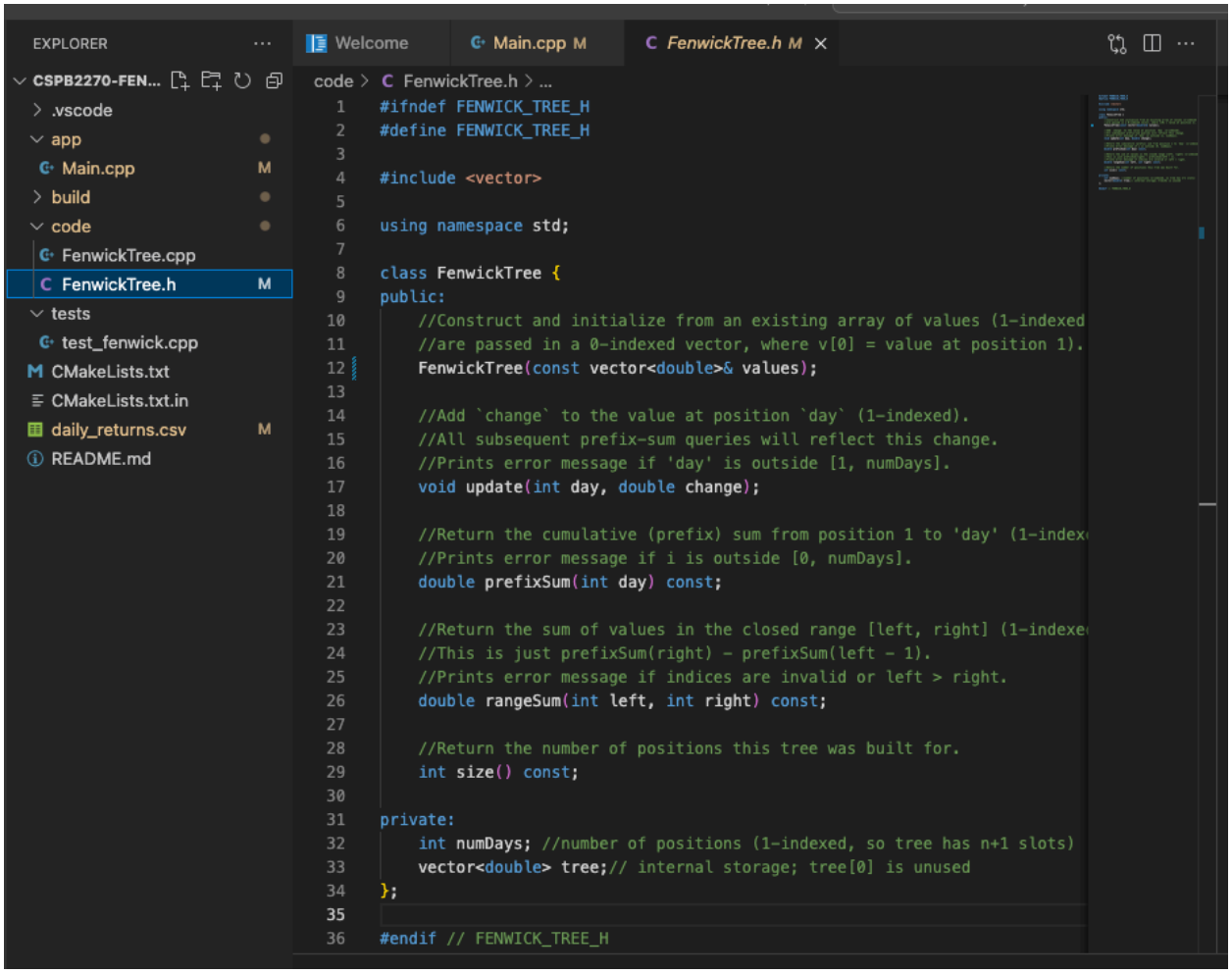
```
EXPLORER
CSPB2270-FENWICKTREESFORI...
  .vscode
  app
    Main.cpp                M
  build
  code
    FenwickTree.cpp
    FenwickTree.h           M
  tests
    test_fenwick.cpp
  CMakeLists.txt
  CMakeLists.txt.in
  daily_returns.csv         M
  README.md
```

Welcome | Main.cpp M

app > Main.cpp > main()

```cpp
vector<double> loadCSV(const string& filename) {

int main() {
    // Load returns from CSV and build the Fenwick Tree
    vector<double> dailyReturns = loadCSV("../daily_returns.csv");
    if (dailyReturns.empty()) {
        cout << "Error: No data found" << endl;
        return 1;
    }

    FenwickTree tree(dailyReturns);
    cout << "Loaded " << dailyReturns.size() << " days of returns.\n";

    //Functionality 1: Range query: cumulative return from day 10 to day 5
    int startDay = 10;
    int endDay   = 50;
    cout << "Cumulative return from day " << startDay << " to day " << endl
         << tree.rangeSum(startDay, endDay) << "\n";

    //Functionality 2: Update: correct day 15's return from +$1450 to -$25
    int correctedDay = 15;
    double originalReturn = dailyReturns[correctedDay - 1];
    double correctedReturn = -2500;
    double delta = correctedReturn - originalReturn;

    tree.update(correctedDay, delta);
    cout << "Updated day " << correctedDay << " from $" << originalReturn
         << " to $" << correctedReturn << "\n";

    //Run again to show the update was reflected
    cout << "Cumulative return from day 10 to day 50 after update: $"
         << tree.rangeSum(startDay, endDay) << "\n";

    return 0;
}
```

The header file gives an overview of each of the class public and private functions, just as in the homework:



The main functions are:

- The Constructor
  - Creates the FenwickTree structure based on the csv-originated values that are passed to it
- The Update function
  - Updates the dollar return for a specified day all the way throughout the tree
- PrefixSum
  - Sums the total cumulative returns from day 1 to the specified date
  - This function is crucially important to RangeSum, because cumulative sums across ranges can just be calculated as the differences between cumulative sums at two different points
- And RangeSum
  - Sums cumulative returns from a specified start and end date

FenwickTree():

```cpp
#include "FenwickTree.h"
#include <iostream>

using namespace std;

//Fenwick Tree Implementation

FenwickTree::FenwickTree(const vector<double>& values) {
    numDays = (static_cast<int>(values.size()));//static cast to convert to int
    tree = vector<double>(numDays + 1, 0.0);

    //copy values into tree at 1-based positions, then propagate
    //each node's value to its parent
    for (int i = 1; i <= numDays; ++i) {
        //copy the raw value, ignoring position 0
        tree[i] += values[i - 1];
        //below is the bit trick that ensures propagation of the sum to parents
        int parent = i + (i & (-i));
        if (parent <= numDays) {
            tree[parent] += tree[i]; //add the sum from the previous node to parent
        }
    }
}
```

The constructor creates a tree, as defined in the private functions of the FenwickTree class, and loops each day defined in the input file, adding to tree based on the position defined by the bit-trick explained in the README file. This is the core of the FenwickTree logic.

Update()

```cpp
void FenwickTree::update(int day, double change) {
    //make sure that the input is valid
    if (day < 1 || day > numDays) {
        cout << "Error: day " << day << " is out of range [1, " << numDays << "]." << endl;
        return;
    }

    //walk upward through all nodes whose coverage range includes position i.
    //adding the lowest set bit moves us to the next covering ancestor.
    while (day <= numDays) {
        tree[day] += change;
        day += (day & (-day));   // i += lowest set bit of i
    }
}
```

The update function first does a data validation check to make sure that the day is a valid date in the range of provided returns, and then traverses the tree in O(log n) using the same bit-trick as before applying the **net** change for the updated day all the way up the Fenwick tree. I think it's important to emphasize that we are adding/subtracting the net change only at nodes whose coverage range includes the position being updated. This is another main benefit of the Fenwick Tree. This one was pretty difficult to understand even though the code is fairly simple. The Fenwick Tree gets that down to O(log n) by pre-storing partial sums at strategic positions so only the affected ancestors need updating.

Summations

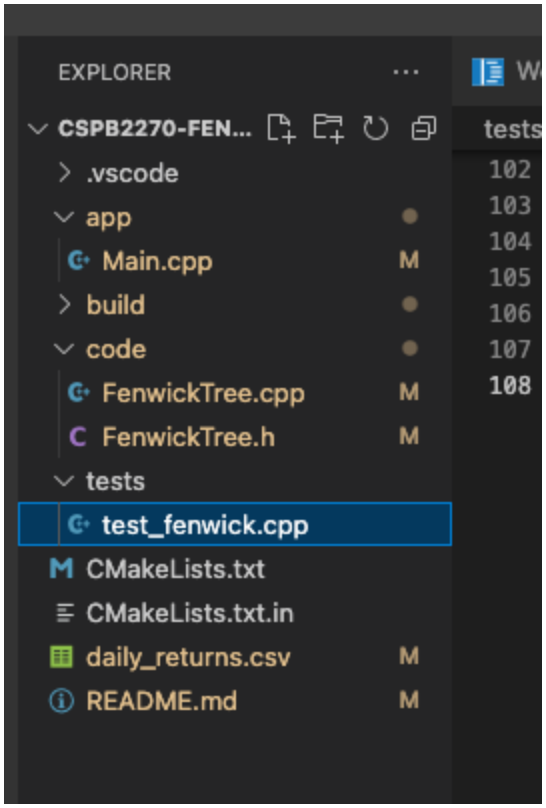
```
double FenwickTree::prefixSum(int day) const {
    //make sure that the input is valid
    if (day < 0 || day > numDays) {
        cout << "Error: day " << day << " is out of range [1, " << numDays << "]." << endl;
        return 0.0;
    }

    //walk downward, accumulating partial sums stored at each node.
    //stripping the lowest set bit moves us to the next node below us.
    double sum = 0.0;
    while (day > 0) {
        sum += tree[day];
        day -= (day & (-day));   // day -= lowest set bit of day
    }
    return sum;
}

double FenwickTree::rangeSum(int left, int right) const {
    //make sure that the input is valid
    if (left < 1 || right > numDays || left > right) {
        cout << "rangeSum(): invalid range [" << left << ", "
            << right << "] for tree of size " << numDays << endl;
        return 0.0;
    }
    //prefixSum(right) includes positions 1 to 'right'
    //prefixSum(left-1) includes positions 1 to 'left-1').
    //their difference is exactly the sum of positions left to right.
    return prefixSum(right) - prefixSum(left - 1);
}
```

As previously mentioned, I implemented two summations to ultimately accomplish rangeSum(). Before the main logic of these functions, I made sure to implement error handling for out-of-range dates. Then prefix sum traverses the tree, creating a cumulative sum up to and including the specified ‘day’ argument. RangeSum() then calculates the sum across the two argument dates by calling prefixSum() on the start and end dates and subtracting them. Very easy!

The above functions are the meat and potatoes of the implementation.

Tests:

As Professor Das encouraged in Piazza, I also utilized Claude to write tests for this code, which are stored in ‘test_fenwick.cpp’ in the tests folder. I had to guide it a little bit on a few of the tests, like including a test that validates that the FenwickTree output is the exact same as a ‘Brute force’ output. However, I think that it did a great job of coming up with edge case tests to make sure that the code is running as expected.



I reused the 'CMakeLists.txt' and 'CMakeLists.txt.in' files from the homework assignments in my repository, and made sure that the executables were looking at the right files so that everything can be ran just as we did for the homeworks.

To run the application, you can use './run_app', and to run the tests, you can use './run_tests' (screenshot below)

```
jovyan@jupyter-daoc2270:~/CSPB2270-FenwickTreesForInvestmentPerformance/build$ make && ./run_app && ./run_tests
Consolidate compiler generated dependencies of target run_tests
[ 50%] Built target run_tests
Consolidate compiler generated dependencies of target run_app
[100%] Built target run_app
Loaded 252 days of returns.
Cumulative return from day 10 to day 50: $12120
Updated day 15 from $1450 to $-2500
Cumulative return from day 10 to day 50 after update: $8170
Running main() from ./googletest/src/gtest_main.cc
[==========] Running 9 tests from 1 test suite.
[----------] Global test environment set-up.
[----------] 9 tests from FenwickTree
[ RUN      ] FenwickTree.StartsAtZero
[       OK ] FenwickTree.StartsAtZero (0 ms)
[ RUN      ] FenwickTree.SingleUpdate
[       OK ] FenwickTree.SingleUpdate (0 ms)
[ RUN      ] FenwickTree.MultipleUpdates
[       OK ] FenwickTree.MultipleUpdates (0 ms)
[ RUN      ] FenwickTree.RangeSum
[       OK ] FenwickTree.RangeSum (0 ms)
[ RUN      ] FenwickTree.BuildFromVector
[       OK ] FenwickTree.BuildFromVector (0 ms)
[ RUN      ] FenwickTree.UpdateModifiesQueries
[       OK ] FenwickTree.UpdateModifiesQueries (0 ms)
[ RUN      ] FenwickTree.NegativeValues
[       OK ] FenwickTree.NegativeValues (0 ms)
[ RUN      ] FenwickTree.MatchesBruteForce
[       OK ] FenwickTree.MatchesBruteForce (0 ms)
[ RUN      ] FenwickTree.OutOfRangeReturnsSafely
Error: day 6 is out of range [1, 5].
rangeSum(): invalid range [3, 1] for tree of size 5
[       OK ] FenwickTree.OutOfRangeReturnsSafely (0 ms)
[----------] 9 tests from FenwickTree (0 ms total)

[----------] Global test environment tear-down
[==========] 9 tests from 1 test suite ran. (0 ms total)
[  PASSED  ] 9 tests.
jovyan@jupyter-daoc2270:~/CSPB2270-FenwickTreesForInvestmentPerformance/build$
```

Other than that, everything is stored in the public repository at this link:
https://github.com/ocon7803/CSPB2270-FenwickTreesForInvestmentPerformance